

ESPRIT

École Supérieure Privée
d'Ingénierie et de Technologies

LinSoft

Société de Services
en Ingénierie Informatique

République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

ESPRIT – École Supérieure Privée d'Ingénierie et de Technologies

Département Génie Logiciel

RAPPORT DE PROJET DE FIN D'ÉTUDES

Automatisation et optimisation de la gestion d'événements via une architecture microservices avec Intelligence Artificielle et déploiement sur OpenShift

Réalisée par : Maryem Achoury

Encadrant académique : [À compléter]

Encadrant professionnel : [À compléter]

Entreprise d'accueil : LinSoft

Spécialité : Génie Logiciel

Année universitaire 2025 – 2026

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous

I validate the submission of the student's report:

Nom & Prénom / Name & Surname : _____

Encadrant Entreprise / Business Supervisor Nom & Prénom : Cachet & Signature : Nom & Prénom : Signature :	Encadrant Académique / Academic Supervisor
---	--

Ce formulaire doit être rempli, signé et scanné / This form must be completed, signed and scanned.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Résumé

Ce projet de fin d'études, mené au sein de la société LinSoft sur une période de six mois, avait pour objectif de pallier les insuffisances du système de gestion d'événements en place – une simple page web statique sans fonctionnalités interactives – en concevant et développant une plateforme complète, intelligente et scalable.

La solution proposée s'appuie sur une **architecture microservices** composée de six services indépendants déployés sur **OpenShift** : *events-service*, *registrations-service*, *users-service*, *notifications-service*, *dashboard-service* et *charges-service*. L'intégrité du système repose sur trois piliers techniques : un bus de messages **Apache Kafka** pour la communication asynchrone inter-services, un serveur d'identité **Keycloak** pour la sécurisation centralisée via OAuth2/OIDC, et une base de données **MongoDB** par service (pattern *Database-per-Service*). Le backend est développé avec **Quarkus** (Java), les interfaces avec **Angular** (BackOffice) et **React** (FrontOffice).

La contribution la plus distinctive de ce projet réside dans le *charges-service*, qui implémente un triple mécanisme : gestion d'un catalogue de ressources, affectation des charges avec calcul automatique du coût prévisionnel, et prédiction du budget global par un modèle **Random Forest** (scikit-learn). Ce double calcul – direct et IA – offre à l'organisateur un outil de pilotage budgétaire sans équivalent dans les solutions du marché.

Mots-clés : Microservices, Quarkus, Angular, React, MongoDB, Apache Kafka, Keycloak, OAuth2/OIDC, OpenShift, Docker, Intelligence Artificielle, Random Forest, Prédiction des coûts, Gestion d'événements.

Abstract

This final-year project, carried out at LinSoft over a six-month internship, aimed at replacing an outdated static event page with a fully featured, intelligent, and cloud-native event management platform. The solution is built on a **microservices architecture** comprising six independently deployable services, orchestrated on **OpenShift** (Red Hat Kubernetes). Asynchronous inter-service communication is handled by **Apache Kafka**, while identity management leverages **Keycloak** (OAuth2/OIDC). Each microservice owns its **MongoDB** instance following the Database-per-Service pattern.

The standout feature is the *charges-service*, which combines a charge catalogue, event-level cost assignment, and an AI-powered **Random Forest** model that predicts total event costs from macroscopic parameters (type, size, duration, location). This dual-validation approach – computed cost vs. predicted cost – enables precise budgetary control that no off-the-shelf solution currently offers.

Keywords: Microservices, Quarkus, Angular, React, MongoDB, Apache Kafka, Keycloak, OAuth2/OIDC, OpenShift, Docker, Artificial Intelligence, Random Forest, Cost Prediction, Event Management.

Dédicace

*À mes parents,
dont le soutien indéfectible a été mon ancre tout au long de ce parcours.*

*À mes frères et sœurs,
pour leur bienveillance et leur encouragement quotidien.*

*À mes amis et collègues de promotion,
avec qui j'ai partagé les moments de doute et de victoire.*

À tous ceux qui croient que la technologie peut rendre le monde plus efficace.

Remerciements

Ce travail n'aurait pas été ce qu'il est sans la contribution de nombreuses personnes que je tiens à remercier sincèrement.

Mes premiers remerciements vont à mon encadrant académique, dont les remarques constructives et la rigueur intellectuelle m'ont aidée à progresser dans la formalisation des concepts et dans la qualité de la rédaction.

Je suis également très reconnaissante envers l'équipe technique de **LinSoft**, et plus particulièrement mon encadrant professionnel, pour la confiance accordée dès le premier jour, la liberté de conception laissée dans les choix d'architecture, et les nombreuses heures de revue de code qui ont fait maturier ce projet.

Je remercie enfin le corps professoral d'ESPRIT pour la formation solide en ingénierie logicielle qui m'a donné les outils conceptuels nécessaires à l'accomplissement de ce travail. La maîtrise des architectures distribuées, des patterns cloud-native et des approches DevOps, acquise lors de mes années d'études, a été déterminante.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
HPA	Horizontal Pod Autoscaler (Kubernetes)
HTTP/S	HyperText Transfer Protocol / Secure
IA	Intelligence Artificielle
IAM	Identity and Access Management
JWT	JSON Web Token
MAE	Mean Absolute Error (erreur absolue moyenne)
ML	Machine Learning
NoSQL	Not Only SQL
OAuth2	Open Authorization 2.0
OIDC	OpenID Connect
REST	Representational State Transfer
RF	Random Forest
RBAC	Role-Based Access Control
SPA	Single Page Application
SSII	Société de Services en Ingénierie Informatique
SSO	Single Sign-On
TDD	Test-Driven Development
TLS	Transport Layer Security
UML	Unified Modeling Language

Table des matières

Résumé	i
Abstract	ii
Dédicace	iii

Remerciements	iv
Liste des abréviations	v
Introduction générale	1
1 Étude Préalable	2
Introduction	2
1.1 Présentation de LinSoft	2
1.2 Problématique	3
1.3 Étude de l'existant	3
1.3.1 Le système actuel de LinSoft	3
1.3.2 Analyse des solutions du marché	4
1.3.3 Synthèse des besoins identifiés	5
1.4 Solution proposée	5
1.4.1 Vision générale	5
1.4.2 Microservices de la plateforme	6
1.4.3 Méthodologie de développement	6
1.5 Environnement de travail	7
1.5.1 Environnement matériel	7
1.5.2 Environnement logiciel	8
Conclusion	10
2 Spécification des Besoins	11
Introduction	11
2.1 Identification des acteurs	11
2.2 Besoins fonctionnels	12
2.2.1 Authentification et gestion des comptes	12
2.2.2 Gestion des événements	12
2.2.3 Gestion des inscriptions	12
2.2.4 Notifications automatisées	12
2.2.5 Tableau de bord analytique	13
2.2.6 Gestion des charges et prédiction des coûts	13
2.3 Besoins non-fonctionnels	15
2.4 Diagrammes de cas d'utilisation	15
2.4.1 Vue globale	15
2.4.2 Zoom sur le charges-service	16
2.5 Diagramme de classes	17
Conclusion	17
3 Conception	18
Introduction	18
3.1 Architecture globale	18

3.1.1	Justification des choix technologiques	18
3.2	Stratégie de communication	19
3.2.1	Communication synchrone : REST/HTTP	19
3.2.2	Communication asynchrone : Apache Kafka	19
3.2.3	Organisation des topics Kafka	20
3.3	Sécurisation via Keycloak	20
3.3.1	Qu'est-ce que Keycloak ?	20
3.3.2	Le JSON Web Token (JWT)	20
3.3.3	Contrôle d'accès basé sur les rôles (RBAC)	21
3.4	Diagrammes de séquence	22
3.5	Diagrammes d'activité	24
3.6	Diagramme de composants	25
3.7	Diagramme d'objets	26
	Conclusion	26
4	Réalisation	27
	Introduction	27
4.1	Environnement de développement	27
4.2	Implémentation	28
4.2.1	APIs REST du charges-service	28
4.2.2	Modèle Random Forest	28
4.3	Validation par captures d'écran	29
4.4	Déploiement sur OpenShift	31
	Conclusion	31
	Conclusion Générale	32

Liste des figures

1.1	Logo de la société LinSoft	2
1.2	Capture de la page événements existante – linsoft.com/fr/events	4
1.3	Cycle de vie du processus Scrum	7
2.1	Diagramme de cas d'utilisation global – Plateforme LinSoft Events	16
2.2	Diagramme de cas d'utilisation – charges-service	16
2.3	Diagramme de classes – Modèle métier complet	17
3.1	Architecture globale de la plateforme	18
3.2	Diagramme de séquence – Inscription à un événement	22
3.3	Diagramme de séquence – Gestion des charges	23

3.4	Diagramme de séquence – Prédiction IA	23
3.5	Diagramme d’activité – Cycle de vie d’un événement	24
3.6	Diagramme d’activité – Processus d’inscription	25
3.7	Diagramme de composants – Architecture interne des microservices	25
3.8	Diagramme d’objets – Instance “DevOps Summit Tunis 2026”	26
4.1	Kafka UI – Topics métier de la plateforme	29
4.2	Kafka UI – Groupes de consommateurs actifs	30
4.3	Diagramme de déploiement – Namespace OpenShift linsoft-events	31

Liste des tableaux

1.1	Analyse comparative des solutions du marché	4
1.2	Besoins identifiés et solutions cibles	5
1.3	Description des six microservices	6
1.4	Planification des sprints Scrum	7
1.5	Caractéristiques du poste de développement	8
1.6	Technologies et outils de développement	9
2.1	Acteurs du système et leurs périmètres d’action	11
2.2	Features du modèle Random Forest	14
2.3	Exigences non-fonctionnelles de la plateforme	15
3.1	Stratégie de communication par cas d’usage	19
3.2	Topics Kafka – Producteurs et consommateurs	20
4.1	Stack technique de la plateforme	27
4.2	APIs REST du charges-service	28

Introduction générale

La numérisation des processus organisationnels constitue aujourd'hui l'un des axes stratégiques prioritaires des entreprises technologiques. Dans un contexte où la vélocité, la traitement des données en temps réel et la résilience des systèmes sont devenus des critères de compétitivité, les architectures logicielles traditionnelles montrent leurs limites face aux exigences des usages modernes.

La gestion des événements professionnels – conférences, workshops, forums – illustre parfaitement cette tension entre les pratiques héritées et les besoins contemporains. Chez LinSoft, une SSII tunisienne active dans l'intégration de solutions cloud et l'organisation d'événements IT régionaux, cette problématique s'est manifestée de manière concrète : absence de système d'inscription en ligne, gestion des ressources matérielles entièrement manuelle, aucun outil de projection budgétaire, et des notifications assurées manuellement par l'équipe.

C'est dans ce cadre que ce projet de fin d'études a été initié. L'objectif était ambitieux : concevoir, développer et déployer une plateforme complète de gestion d'événements, en adoptant les standards actuels de l'ingénierie cloud – architecture microservices, communication asynchrone via Apache Kafka, sécurisation par Keycloak, persistance distribuée MongoDB – et en y intégrant un module d'intelligence artificielle pour la prédiction des coûts logistiques.

Le présent rapport est structuré en quatre chapitres complémentaires :

- Le **Chapitre 1 – Étude préalable** – décrit l'organisme d'accueil, pose la problématique, analyse les solutions existantes sur le marché et détaille la solution retenue avec sa méthodologie de développement.
- Le **Chapitre 2 – Spécification des besoins** – présente les acteurs du système, les exigences fonctionnelles et non-fonctionnelles, accompagnées des diagrammes de cas d'utilisation et du modèle de classes.
- Le **Chapitre 3 – Conception** – détaille l'architecture microservices, les choix de communication, la modélisation des flux métier et la sécurisation, illustrés par l'ensemble des diagrammes UML.
- Le **Chapitre 4 – Réalisation** – couvre l'environnement technique, l'implémentation des services, les interfaces utilisateur et le déploiement sur OpenShift avec les éléments de validation.

Chapitre 1

Étude Préalable

Introduction

Tout projet de développement logiciel sérieux commence par une phase d'analyse approfondie du contexte : comprendre l'entreprise qui accueille le stagiaire, identifier avec précision les dysfonctionnements existants, et évaluer les solutions disponibles sur le marché avant de formuler une proposition adaptée. Ce chapitre parcourt ces étapes fondatrices.

1.1 Présentation de LinSoft

LinSoft est une Société de Services en Ingénierie Informatique (SSII) fondée en Tunisie, spécialisée dans l'intégration de solutions IT complexes pour des clients de la région Moyen-Orient et Afrique (MEA). Positionnée sur le segment haut de gamme, elle intervient dans des domaines à forte valeur technique :

- Conception et développement d'applications web et mobiles ;
- Architecture microservices, systèmes distribués et cloud-native ;
- Déploiement sur plateformes conteneurisées (OpenShift, Kubernetes) ;
- Intégration IAM, sécurisation et gestion des identités ;
- Solutions de data engineering et d'intelligence artificielle ;
- Formation et certification aux technologies stratégiques (Quarkus, Kafka, Red Hat).



Figure 1.1: Logo de la société LinSoft

LinSoft organise régulièrement des événements IT d’envergure – conférences techniques, workshops de certification, participation au GITEX Afrique – qui rassemblent des centaines de participants. Ces événements, essentiels pour la visibilité et le rayonnement de l’entreprise, étaient pourtant gérés avec des outils n’étant pas à la hauteur des ambitions de la société.

1.2 Problématique

L’observation du fonctionnement interne de LinSoft a permis d’identifier plusieurs dysfonctionnements concrets dans la gestion de ses événements :

1. **Absence totale d’inscription en ligne** : les participants devaient contacter LinSoft par email ou téléphone, générant un traitement manuel fastidieux et source d’erreurs.
2. **Aucun suivi en temps réel** : aucun outil ne permettait de connaître instantanément le nombre d’inscrits, les listes d’attente, ou le taux de remplissage.
3. **Notifications manuelles** : les confirmations d’inscription et les rappels étaient envoyés manuellement, sans cohérence ni automatisation.
4. **Gestion des ressources absente** : les besoins matériels (projecteurs, tables, traiteur, microphones) étaient estimés à la louche, sans catalogue ni traitement structuré.
5. **Budget empirique** : sans outil de calcul, les coûts étaient sous-estimés ou sur-estimés, rendant le pilotage budgétaire impossible.

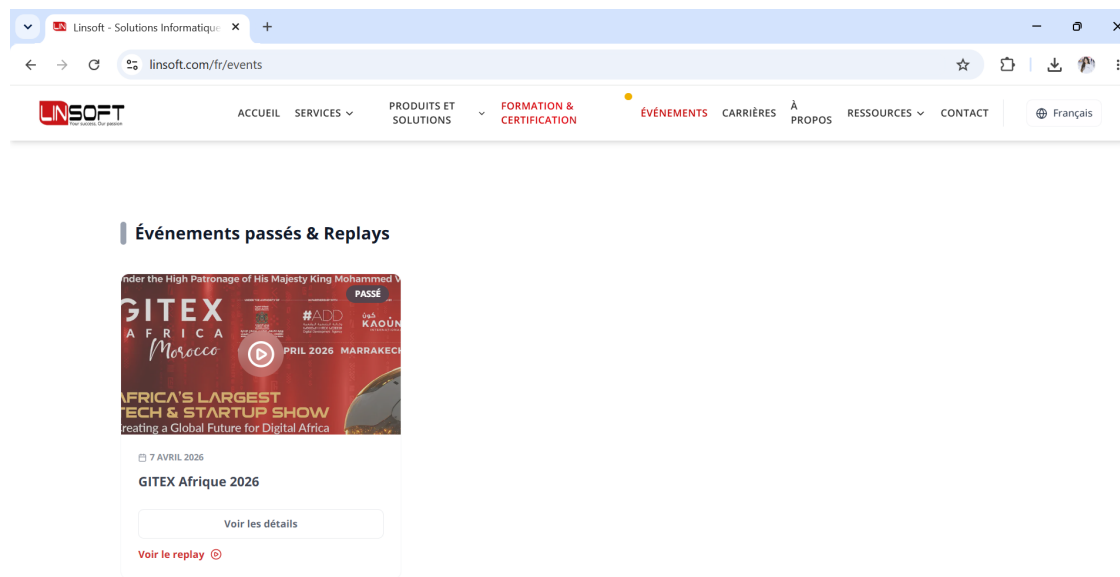
Ces lacunes engendraient des pertes de temps significatives, des risques d’erreurs humaines répétées et une incapacité à monter en charge lors des grands événements. La nécessité d’une solution numérique intégrée s’est imposée comme une priorité opérationnelle.

1.3 Étude de l’existant

1.3.1 Le système actuel de LinSoft

LinSoft disposait, au moment de démarrer ce projet, d’une page dédiée aux événements sur son site institutionnel (linsoft.com/fr/events), sous le slogan “*Inspirez-vous. Connectez-vous. Évoluez.*”. Cette page se limitait à lister les événements passés et à venir, sans aucune interaction possible :

- **Pas d’inscription** : aucun formulaire, aucun bouton d’action ;
- **Pas de gestion** : les événements étaient créés directement dans le CMS, sans workflow d’approbation ni gestion de statut ;
- **Pas de données** : LinSoft ne disposait d’aucune donnée structurée sur ses participants, leur provenance ou leur comportement.

Figure 1.2: Capture de la page événements existante – `linsoft.com/fr/events`

1.3.2 Analyse des solutions du marché

Plusieurs plateformes existent pour la gestion d'événements. Leur étude comparée a mis en évidence des limitations dans le contexte spécifique de LinSoft :

Tableau 1.1: Analyse comparative des solutions du marché

Solution	Points forts	Limites pour LinSoft
Eventbrite	Inscription en ligne, paiement	Pas de module charges/IA, RGPD non adapté, coût élevé
Meetup	Communautés, récurrence	Orienté grand public, RBAC limité, pas de gestion budgétaire
Google Forms	Simplicité, gratuit	Pas de gestion de capacité, pas d'automatisation, no tracking
Développement maison antérieur	Personnalisable	Monolithique, non scalable, maintenance difficile
Notre solution	Microservices, IA, Kafka, OpenShift	Spécifique LinSoft, entièrement personnalisée

Cette comparaison justifie le choix d'un développement sur mesure : aucune solution existante ne couvrirait simultanément la gestion des charges matérielles, la prédiction des coûts par IA, et la scalabilité cloud requise.

1.3.3 Synthèse des besoins identifiés

Tableau 1.2: Besoins identifiés et solutions cibles

Besoin	Situation actuelle	Solution cible
Inscription en ligne	Canal externe (email)	Formulaire + confirmation automatique
Suivi participants	Non centralisé	Dashboard temps réel (Kafka)
Notifications	Manuelles	Email/SMS automatisés via Kafka
Gestion des charges	Absente	Catalogue + affectation + total
Prédiction coûts	Empirique	Random Forest scikit-learn
Scalabilité	Site monolithique	Microservices + OpenShift HPA
Sécurité	Basique	Keycloak OAuth2/OIDC, JWT

1.4 Solution proposée

1.4.1 Vision générale

La solution proposée est une plateforme cloud-native de gestion d'événements, conçue autour de six microservices autonomes communiquant via REST (synchrone) et Apache Kafka (asynchrone). Elle repose sur les principes suivants :

- **Séparation des responsabilités** (Single Responsibility) : chaque service couvre un domaine métier précis et déployé indépendamment ;
- **Base de données par service** (Database-per-Service) : chaque microservice possède son instance MongoDB, garantissant l'isolation totale des données ;
- **Communication pilotée par événements** (Event-Driven) : les actions métier génèrent des événements Kafka consommés de manière asynchrone ;
- **Sécurisation centralisée** : un serveur Keycloak unique gère l'authentification et les autorisations pour tous les services.

1.4.2 Microservices de la plateforme

Tableau 1.3: Description des six microservices

Microservice	Port	Responsabilités
events-service	8081	Création, publication, modification et cycle de vie des événements
registrations-service	8082	Inscriptions, listes d’attente, gestion des places disponibles
users-service	8083	Comptes utilisateurs, profils, synchronisation avec Keycloak
notifications-service	8084	Email et SMS automatisés, pilotés par consommation Kafka
dashboard-service	8085	Agrégation de statistiques en temps réel, exports
charges-service	8086	Catalogue de charges, affectation, calcul budgétaire et prédiction IA

1.4.3 Méthodologie de développement

Méthode de gestion de projet : Agile Scrum

Pour réaliser notre projet, nous avons choisi d’utiliser le cadre de travail **SCRUM**, dont la structure favorise les réunions quotidiennes (*Daily Stand-up*), ce qui renforce la communication entre les membres de l’équipe et contribue à instaurer une atmosphère de travail collaborative. Cette dynamique de groupe augmente le rendement collectif et, par conséquent, accélère l’avancement du projet tout en améliorant la qualité livrée.

En outre, SCRUM nous a permis de mieux respecter les délais planifiés pour chacune des parties du projet. Son principe central consiste à concentrer l’équipe de développement sur un ensemble de fonctionnalités à réaliser de façon itérative, dans des itérations de deux semaines appelées **Sprints**. Chaque Sprint aboutit à la livraison d’un incrément fonctionnel, testable et intégrable dans la solution globale.

Déroulement des sprints. Le backlog produit a été maintenu sur **Jira**, avec des déploiements continus via un pipeline CI/CD sur **OpenShift**. Chaque sprint délivrait un microservice fonctionnel et intégré, ce qui a permis de valider les choix d’architecture au fil du développement plutôt que d’en découvrir les problèmes en fin de projet.

Le tableau ci-dessous présente la répartition des sprints et leurs livrables respectifs :

Tableau 1.4: Planification des sprints Scrum

Sprint	Période	Livrables
S1	Semaines 1–2	Infrastructure Docker, Keycloak, API Gateway, events-service (CRUD)
S2	Semaines 3–4	registrations-service, gestion des places et liste d'attente
S3	Semaines 5–6	users-service, synchronisation Keycloak, RBAC
S4	Semaines 7–8	notifications-service, topics Kafka, email/SMS automatisés
S5	Semaines 9–10	charges-service, catalogue, affectation, calcul budgétaire
S6	Semaines 11–12	Prédiction IA (Random Forest), dashboard-service, FrontOffice React
S7	Semaines 13–14	BackOffice Angular, déploiement OpenShift, tests end-to-end

La figure ci-dessous illustre les étapes à suivre dans le cycle de vie du SCRUM :

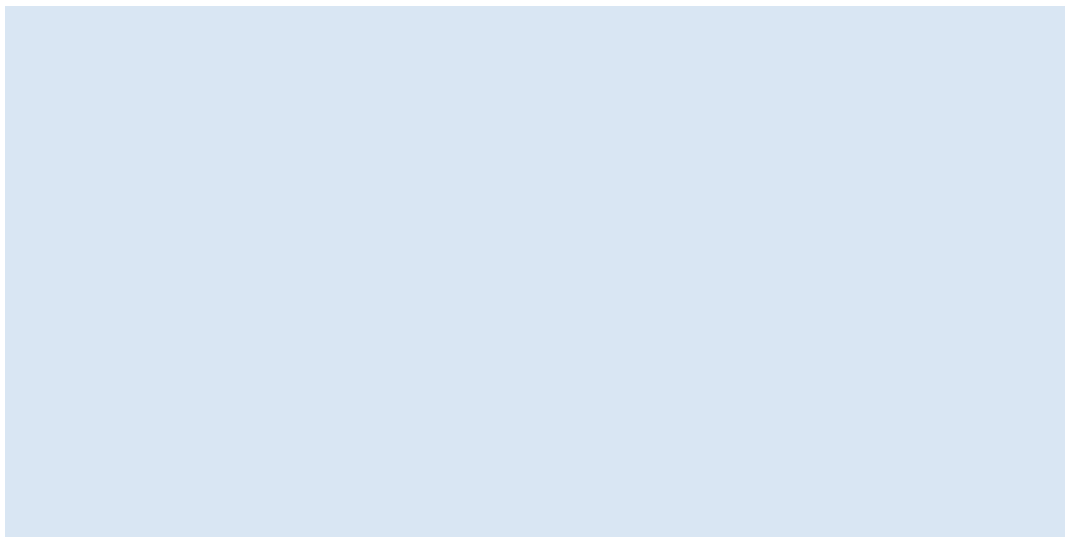


Figure 1.3: Cycle de vie du processus Scrum

1.5 Environnement de travail

1.5.1 Environnement matériel

Le développement du projet a été réalisé sur le poste de travail suivant :

Tableau 1.5: Caractéristiques du poste de développement

Composant	Spécification
Ordinateur	Lenovo IdeaPad Flex 5
Processeur	Intel Core i5 – 11 ^e génération
Mémoire RAM	8 Go DDR4
Stockage	SSD 512 Go
Système	Windows 11 / WSL2 Ubuntu 22.04

1.5.2 Environnement logiciel

Le tableau ci-dessous présente l'ensemble des technologies et outils utilisés pour concevoir, développer et déployer la plateforme. Chaque choix est motivé par des critères techniques adaptés aux contraintes du projet.

Tableau 1.6: Technologies et outils de développement

Technologie	Description	Logo
Quarkus 3.x	Framework Java conçu pour le cloud natif. Il offre un démarrage ultra-rapide, une empreinte mémoire réduite et des extensions prêtes à l'emploi pour Kafka, MongoDB et Keycloak.	
MongoDB 6.x	Base de données NoSQL orientée documents. Son schéma flexible permet d'adapter les structures de données sans migration, et sa réplication native garantit la haute disponibilité.	
Apache Kafka 3.x	Plateforme de messagerie distribuée. Elle sert de bus d'événements asynchrone entre les microservices, avec garantie de livraison et conservation ordonnée des messages.	
Keycloak 22.x	Serveur open source de gestion des identités (IAM). Il implémente OAuth2 et OpenID Connect pour l'authentification centralisée et le contrôle d'accès basé sur les rôles (RBAC).	
scikit-learn 1.3	Bibliothèque Python de machine learning. Elle fournit l'algorithme Random Forest utilisé pour la prédiction des coûts d'événements, ainsi que les outils de validation croisée et de sérialisation.	
Angular 17.x	Framework TypeScript pour le développement d'applications web mono-page. Utilisé pour le BackOffice destiné aux organisateurs et administrateurs.	
React 18.x	Bibliothèque JavaScript pour la construction d'interfaces utilisateur réactives. Utilisée pour le FrontOffice accessible aux participants.	
Docker 24.x	Plateforme de conteneurisation. Elle encapsule chaque microservice dans un conteneur isolé, garantissant la reproductibilité de l'environnement entre le développement et la production.	

Conclusion

Ce premier chapitre a posé le cadre du projet : l'analyse de l'existant a révélé des lacunes significatives dans la gestion des événements LinSoft, que les solutions du marché ne pouvaient combler. L'environnement de travail et la pile technologique ont été sélectionnés pour répondre aux exigences de scalabilité et de sécurité identifiées. Le chapitre suivant précise les besoins fonctionnels et non-fonctionnels de la plateforme.

Chapitre 2

Spécification des Besoins

Introduction

La spécification des besoins constitue le pont entre l'analyse du problème et sa conception technique. Elle formalise ce que le système doit faire (besoins fonctionnels), les contraintes de qualité qu'il doit respecter (besoins non-fonctionnels), et les acteurs qui interagiront avec lui. Ce chapitre présente également les diagrammes de cas d'utilisation et le modèle de classes qui résultent de cette analyse.

2.1 Identification des acteurs

Trois catégories d'acteurs humains interagissent avec la plateforme, auxquels s'ajoute le système Keycloak comme acteur technique.

Tableau 2.1: Acteurs du système et leurs périmètres d'action

Acteur	Rôle et périmètre d'action
Participant	Utilisateur final qui consulte les événements publiés, s'y inscrit, gère ses inscriptions et reçoit des notifications automatiques. Son accès est limité aux fonctionnalités publiques.
Organisateur	Professionnel LinSoft chargé de créer et gérer des événements. Il dispose d'un accès au BackOffice pour éditer des événements, affecter des charges et consulter les prédictions IA.
Administrateur	Gestionnaire de la plateforme avec accès complet : gestion des utilisateurs, attribution des rôles, supervision globale et accès à toutes les statistiques.
Keycloak (Syst.)	Serveur d'identité qui émet et valide les tokens JWT pour toutes les requêtes traversant l'API Gateway.

2.2 Besoins fonctionnels

2.2.1 Authentification et gestion des comptes

La sécurité est le premier niveau du système. Tout accès à une fonctionnalité protégée passe par une authentification auprès de Keycloak, qui délivre un token JWT signé. Les besoins couverts sont :

- Authentification par identifiants via SSO Keycloak (realm `linsoft-events`) ;
- Gestion du profil utilisateur (nom, email, mot de passe) ;
- Attribution de rôles (`ROLE_ADMIN`, `ROLE_ORGANIZER`, `ROLE_PARTICIPANT`) avec accès différencié ;
- Invalidation de session et renouvellement de token (refresh token).

2.2.2 Gestion des événements

Le cœur de la plateforme concerne la création et le suivi du cycle de vie des événements. Un événement passe par les états : BROUILLON → PUBLIÉ → TERMINÉ ou ANNULÉ. Les fonctions couvertes incluent :

- Création d'un événement avec tous ses attributs (titre, description, date, lieu, capacité maximale, budget estimé) ;
- Publication visible par les participants dès validation du budget ;
- Modification d'un événement avec notification automatique aux inscrits ;
- Annulation déclenchant une notification Kafka à tous les participants concernés ;
- Recherche et filtrage par date, lieu, statut ou catégorie.

2.2.3 Gestion des inscriptions

Le service d'inscriptions gère la relation entre participants et événements avec une logique de liste d'attente automatique :

- Vérification de la capacité disponible avant toute inscription ;
- Création d'une inscription avec statut `CONFIRMÉE` si des places sont disponibles, `EN_ATTENTE` sinon ;
- Promotion automatique du premier en liste d'attente lors d'une annulation ;
- Consultation des inscriptions de l'utilisateur connecté.

2.2.4 Notifications automatisées

Les notifications ne sont pas déclenchées manuellement mais pilotées par la consommation des topics Kafka. Aucun code de notification n'est présent dans les autres services :

- Email de confirmation dès validation de l'inscription ;
- Rappels automatiques J-7 et J-1 avant la date de l'événement ;

- Notification d’annulation ou de modification majorée ;
- Notification de promotion depuis la liste d’attente.

2.2.5 Tableau de bord analytique

Le dashboard agrège en temps réel les données issues de Kafka pour fournir :

- Nombre d’événements par statut, inscrits par événement, taux de remplissage ;
- Évolution temporelle des inscriptions ;
- Export CSV/PDF des rapports statistiques.

2.2.6 Gestion des charges et prédiction des coûts

Le *charges-service* est la contribution technique la plus originale de ce projet. Il s’organise en trois niveaux fonctionnels complémentaires et progressifs.

Niveau 1 – Gestion du catalogue de charges

Un catalogue centralisé répertorie toutes les ressources matérielles et services récurrents des événements LinSoft. Chaque **Charge** est caractérisée par : un libellé descriptif (ex. : *Projecteur Full HD*, *Stylo personnalisé LinSoft*, *Pause-café par personne*), une **catégorie** (EQUIPEMENT, LOGISTIQUE, RESTAURATION, SECURITE), un **prix unitaire** en TND, et une description optionnelle.

Les opérations disponibles sont le CRUD complet : création, lecture paginée et filtrable, modification du prix unitaire, et suppression sous contrôle (impossible si la charge est affectée à un événement actif).

Niveau 2 – Affectation des charges à un événement

L’organisateur compose le budget prévisionnel de son événement en sélectionnant des charges du catalogue et en renseignant les quantités. Le système calcule automatiquement :

$$\text{sousTotal}_i = \text{prixUnitaire}_i \times \text{quantité}_i \quad (2.1)$$

$$\text{coûtTotal} = \sum_{i=1}^n \text{sousTotal}_i \quad (2.2)$$

Exemple concret pour un événement de 200 participants :

Projecteur ($150 \times 3 = 450$ TND) + Stylo ($1.5 \times 200 = 300$ TND) + Traiteur ($35 \times 200 = 7000$ TND) + Micro ($80 \times 2 = 160$ TND) = **7910** TND

Niveau 3 – Prédiction des coûts par Intelligence Artificielle

En complément du calcul direct – qui requiert que toutes les charges aient déjà été identifiées et renseignées – le modèle IA permet d’obtenir une **estimation en amont**, dès la phase de planification, à partir de paramètres macroscopiques :

Tableau 2.2: Features du modèle Random Forest

Feature	Type	Description
typeEvenement	Catégoriel	Conference, workshop, seminaire, séminaire...
nbParticipants	Numérique	Nombre de participants attendus
dureeHeures	Numérique	Durée totale de l’événement en heures
localisation	Catégoriel	Ville et pays d’accueil
has_equipement	Booléen	Présence de charges de type EQUIPEMENT
has_logistique	Booléen	Présence de charges de type LOGISTIQUE
has_restauraton	Booléen	Présence de charges de type RESTAURATION
coutTotal (target)	Numérique	Coût total réel de l’événement

Le modèle retourne un coût estimé, un niveau de confiance et un intervalle de prédiction, permettant une double validation : le coût calculé depuis les affectations sert de référence exacte, tandis que la prédiction IA alerte l’organisateur si des charges semblent manquantes (écart significatif entre les deux valeurs).

2.3 Besoins non-fonctionnels

Tableau 2.3: Exigences non-fonctionnelles de la plateforme

Critère	Exigence détaillée
Performance	Temps de réponse REST < 2 secondes en charge normale ; débit Kafka > 10 000 messages/seconde.
Disponibilité	Objectif de 99,9% via réplication multi-pods sur OpenShift (Horizontal Pod Autoscaler activé).
Sécurité	Authentification OAuth2/OIDC, tokens JWT signés RS256, HTTPS (TLS 1.3) sur toutes les routes publiques, secrets OpenShift chiffrés.
Scalabilité	Chaque microservice scalable indépendamment selon sa charge propre ; architecture stateless facilitant la mise à l'échelle horizontale.
Maintenabilité	Documentation Swagger/OpenAPI par service, couverture de tests unitaires > 80%, pipeline CI/CD automatisé.
Portabilité	Conteneurisation Docker garantissant la reproductibilité entre environnements (local, staging, production).

2.4 Diagrammes de cas d'utilisation

2.4.1 Vue globale

Le diagramme de la figure 2.1 représente l'ensemble des cas d'utilisation de la plateforme, répartis selon les trois acteurs. La disposition horizontale (*left to right*) permet de lire clairement les zones d'accès de chaque acteur sans croisement de connexions. On note que l'authentification est un prérequis commun à tous, géré de manière transparente par Keycloak.



Figure 2.1: Diagramme de cas d'utilisation global – Plateforme LinSoft Events

2.4.2 Zoom sur le charges-service

Le charges-service mérite un diagramme dédié (figure 2.2) en raison de la sophistication de ses interactions. L'organisateur pilote les trois niveaux (catalogue, affectation, prédiction), tandis que l'Administrateur dispose d'un droit supplémentaire d'administration globale du catalogue. Le Système IA est représenté comme acteur secondaire déclenché lors de la requête de prédiction.



Figure 2.2: Diagramme de cas d'utilisation – charges-service

2.5 Diagramme de classes

Le modèle de classes (figure 2.3) formalise les entités métier et leurs relations. On y retrouve les huit classes principales du système : User, Event, Registration, Notification, Charge, ChargeEvent, CoutPrediction et DashboardStat, ainsi que les énumérations associées (RoleEnum, StatutEvent, StatutInscription, CategorieCharge...).

La relation clef du module de charges est la classe d'association ChargeEvent qui matérialise le lien entre une Charge du catalogue et un Event, portant la quantité affectée et le sous-total calculé automatiquement.

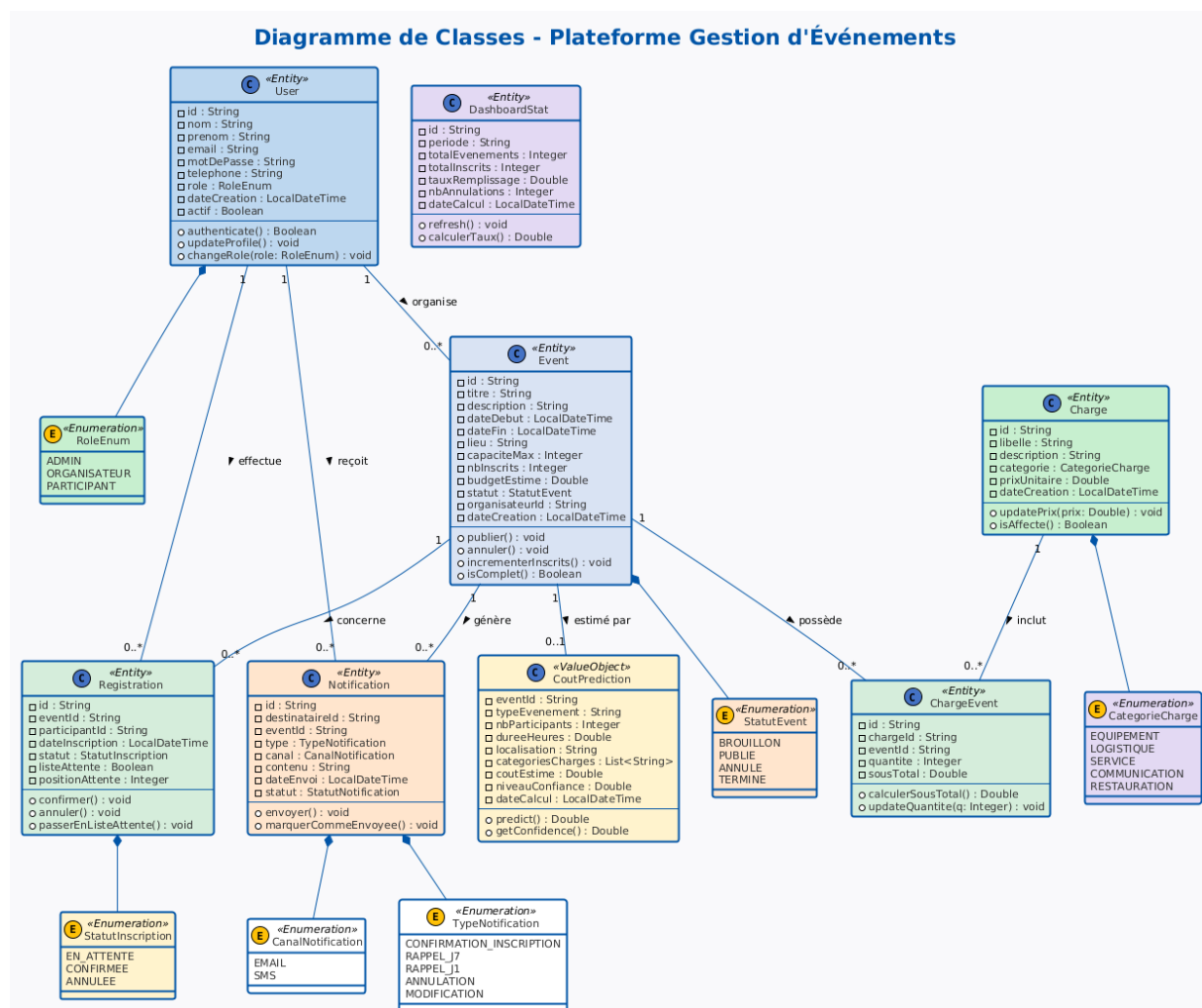


Figure 2.3: Diagramme de classes – Modèle métier complet

Conclusion

Ce chapitre a défini avec précision ce que le système doit réaliser. La richesse fonctionnelle du *charges-service* – triple mécanisme calcul direct + IA – constitue la valeur ajoutée distinctive de cette plateforme. Le chapitre suivant aborde la conception architecturale.

Chapitre 3

Conception

Introduction

Ce chapitre détaille l'architecture microservices retenue, les patterns de communication adoptés, la sécurisation via Keycloak, et présente l'ensemble des diagrammes UML qui formalisent les flux et structures du système.

3.1 Architecture globale

La plateforme s'organise autour d'un **API Gateway** (port 8080) qui constitue le point d'entrée unique de toutes les requêtes client. Ce composant prend en charge le routage vers les microservices, la validation des tokens JWT et le load balancing. Les deux frontends (FrontOffice React :4300, BackOffice Angular :4200) ne communiquent jamais directement avec les services métier.

Architecture globale – Microservices LinSoft Events

(insérer l'image correspondante)

Figure 3.1: Architecture globale de la plateforme

3.1.1 Justification des choix technologiques

Quarkus a été retenu pour ses performances en contexte conteneurisé : démarrage en millisecondes, empreinte mémoire réduite de 30 à 50% par rapport à Spring Boot. **MongoDB** (NoSQL document) est adapté aux entités à structure variable. **Kafka** a été préféré à RabbitMQ pour sa durabilité, son débit et sa sémantique de log distribué permettant le jeu des événements.

3.2 Stratégie de communication

Dans une architecture microservices, les services ne partagent ni mémoire ni base de données. Ils doivent donc communiquer via un réseau. Deux familles de protocoles ont été retenues selon la nature des interactions.

3.2.1 Communication synchrone : REST/HTTP

REST (*Representational State Transfer*) est un style architectural basé sur le protocole HTTP. Lorsqu'un service envoie une requête REST, il attend immédiatement une réponse – c'est ce qu'on appelle une communication **synchrone**. Cette approche convient aux actions qui requièrent un résultat instantané, comme vérifier si des places sont disponibles avant de confirmer une inscription.

Dans notre plateforme, toutes les requêtes utilisateur transitent par l'**API Gateway** (port 8080), qui les redirige vers le microservice compétent via REST. Cette centralisation simplifie la sécurité et le routage : le client ne connaît qu'un seul point d'entrée.

3.2.2 Communication asynchrone : Apache Kafka

Apache Kafka est une plateforme de messagerie distribuée. Plutôt que d'appeler directement un autre service, un microservice **publie un message** (appelé *événement*) dans un canal thématique nommé **topic**. D'autres services s'abonnent à ce topic et traitent le message à leur propre rythme, sans bloquer l'expéditeur – c'est une communication **asynchrone**.

Cette approche présente plusieurs avantages concrets. D'abord, le **découplage** : si le service de notifications est temporairement hors ligne, les messages s'accumulent dans Kafka et sont traités dès son redémarrage, sans perte de données. Ensuite, la **scalabilité** : plusieurs instances d'un même service peuvent consommer les messages en parallèle. Enfin, la **traçabilité** : Kafka conserve un journal ordonné de tous les événements, ce qui facilite l'audit et le débogage.

Le tableau suivant résume le choix du protocole selon le cas d'usage :

Tableau 3.1: Stratégie de communication par cas d'usage

Protocole	Usage	Exemples concrets
REST (sync.)	Actions nécessitant une réponse immédiate	Création d'événement, inscription, calcul budgétaire, prédiction IA
Kafka (async.)	Actions pouvant être traitées en différé	Envoi d'emails/SMS, mise à jour du dashboard, incrément des compteurs

3.2.3 Organisation des topics Kafka

Dans Kafka, un **topic** est un canal nommé dans lequel sont publiés des messages d'un même type. Notre plateforme définit trois familles de topics, chacune correspondant à un domaine métier. Le service qui publie un message est appelé **producteur** (*producer*) ; le service qui le reçoit et le traite est appelé **consommateur** (*consumer*).

Tableau 3.2: Topics Kafka – Producteurs et consommateurs

Topic	Producteur	Consommateurs
event.created/updated/cancelled	events-service	notifications, dashboard
registration.confirmed/cancelled	registrations-service	notifications, events, dashboard
registration.waitlisted	registrations-service	notifications-service
user.created	users-service	notifications-service

Par exemple, lorsqu'un participant s'inscrit à un événement, le `registrations-service` publie un message `registration.confirmed` dans Kafka. Le `notifications-service` le consomme pour envoyer un email de confirmation, le `dashboard-service` le consomme pour mettre à jour les statistiques, et le `events-service` l'utilise pour décrémenter le nombre de places disponibles. Tout cela se passe en parallèle, sans qu'aucun service n'attende les autres.

3.3 Sécurisation via Keycloak

3.3.1 Qu'est-ce que Keycloak ?

Keycloak est un serveur open source d'**gestion des identités et des accès** (IAM – *Identity and Access Management*), développé par Red Hat. Il joue le rôle d'une **gardienne centralisée** de l'authentification : plutôt que chaque microservice gère ses propres comptes et mots de passe, Keycloak prend en charge cette responsabilité pour l'ensemble de la plateforme.

Keycloak implémente le protocole **OAuth2** et sa surcouche **OpenID Connect (OIDC)**, qui sont des standards industriels pour l'authentification déléguée. En pratique, quand un utilisateur se connecte, il n'envoie ses identifiants qu'à Keycloak. Si l'authentification réussit, Keycloak retourne un **jeton JWT** (*JSON Web Token*) signé, que l'utilisateur présente ensuite à chaque requête.

3.3.2 Le JSON Web Token (JWT)

Un JWT est un jeton numérique compact et autonome. Il contient trois parties encodées en Base64 : l'en-tête (algorithme de signature), le *payload* (informations sur l'utilisateur : identifiant,

rôles, date d'expiration) et la **signature cryptographique**. Cette signature, générée par Keycloak avec une clé privée RSA (algorithme RS256), garantit qu'aucun tiers n'a altéré le jeton.

L'API Gateway vérifie la signature de chaque JWT entrant en utilisant la clé publique de Keycloak. Si la vérification échoue ou si le jeton est expiré, la requête est rejetée avant même d'atteindre les microservices. Ce mécanisme évite toute usurpation d'identité.

3.3.3 Contrôle d'accès basé sur les rôles (RBAC)

Le contrôle d'accès basé sur les rôles (RBAC – *Role-Based Access Control*) est un modèle de sécurité où chaque action est autorisée non pas pour un utilisateur précis, mais pour un **rôle**. On assigne un rôle à chaque utilisateur, et ce rôle détermine ce qu'il peut faire dans le système.

Notre plateforme définit un **realm** Keycloak nommé `linsoft-events` (un realm est un espace de configuration isolé) avec trois rôles distincts :

- **ROLE_PARTICIPANT** : peut consulter les événements, s'inscrire, annuler son inscription et recevoir des notifications. C'est le rôle attribué par défaut à tout nouvel utilisateur.
- **ROLE_ORGANIZER** : dispose des droits du participant, plus la capacité de créer, modifier et publier des événements, ainsi que de gérer les charges budgétaires et consulter le tableau de bord analytique.
- **ROLE_ADMIN** : accès complet – gestion des utilisateurs, attribution des rôles, supervision de la plateforme et export des rapports.

Le flux d'authentification complet se déroule comme suit. L'utilisateur saisit ses identifiants dans l'interface (FrontOffice ou BackOffice). Keycloak vérifie les informations et génère un JWT contenant le rôle de l'utilisateur. Ce jeton est transmis à l'API Gateway, qui le valide cryptographiquement. Chaque microservice reçoit ensuite le jeton via l'extension `quarkus-oidc` et vérifie si le rôle présent autorise l'opération demandée. Toute tentative d'accès avec un rôle insuffisant retourne une erreur HTTP 403 (Forbidden).

3.4 Diagrammes de séquence

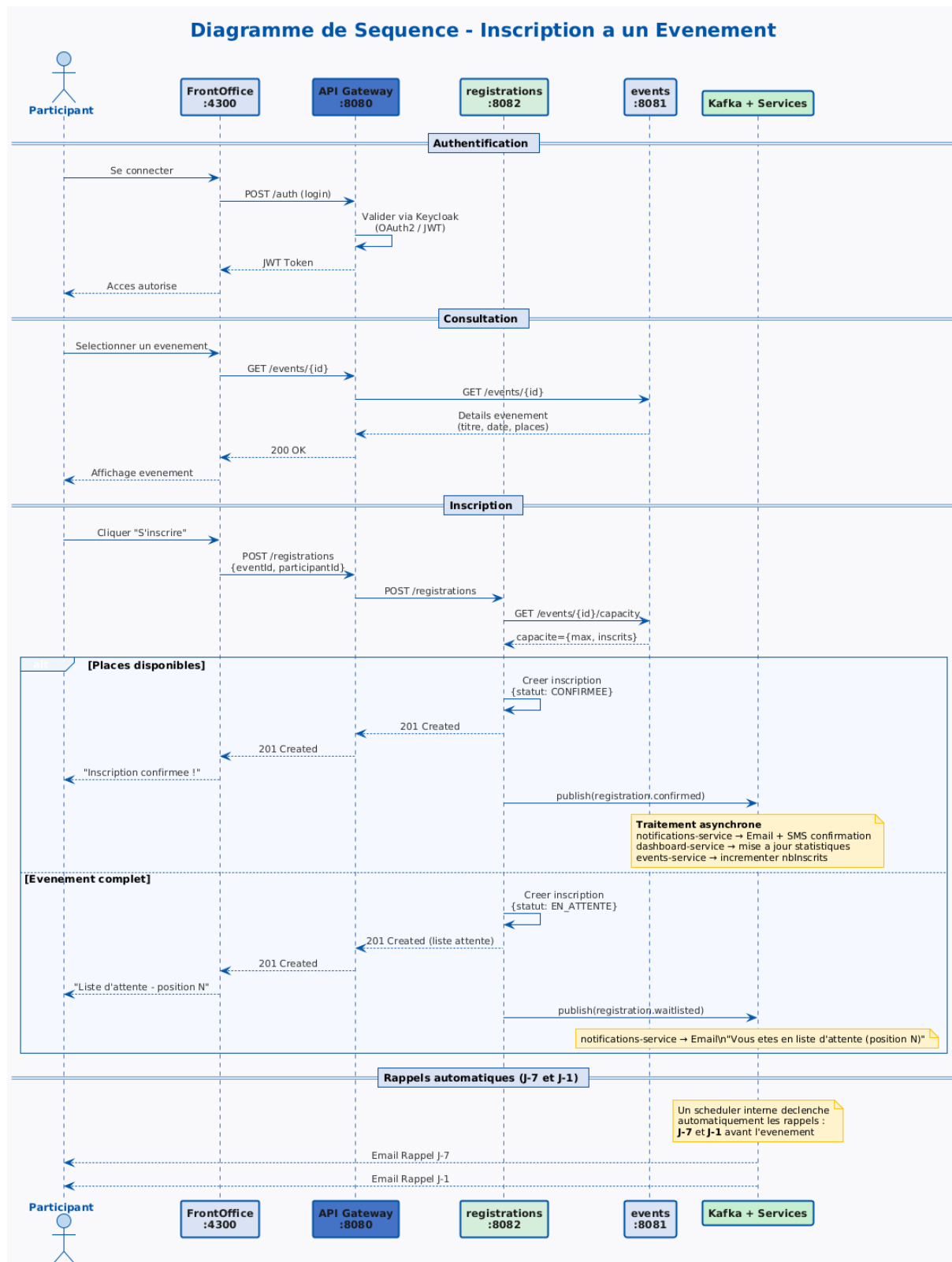


Figure 3.2: Diagramme de séquence – Inscription à un événement

Séquence – Gestion des charges et calcul coût

(insérer l'image correspondante)

Figure 3.3: Diagramme de séquence – Gestion des charges

Séquence – Prédiction des coûts par IA

(insérer l'image correspondante)

Figure 3.4: Diagramme de séquence – Prédiction IA

3.5 Diagrammes d'activité

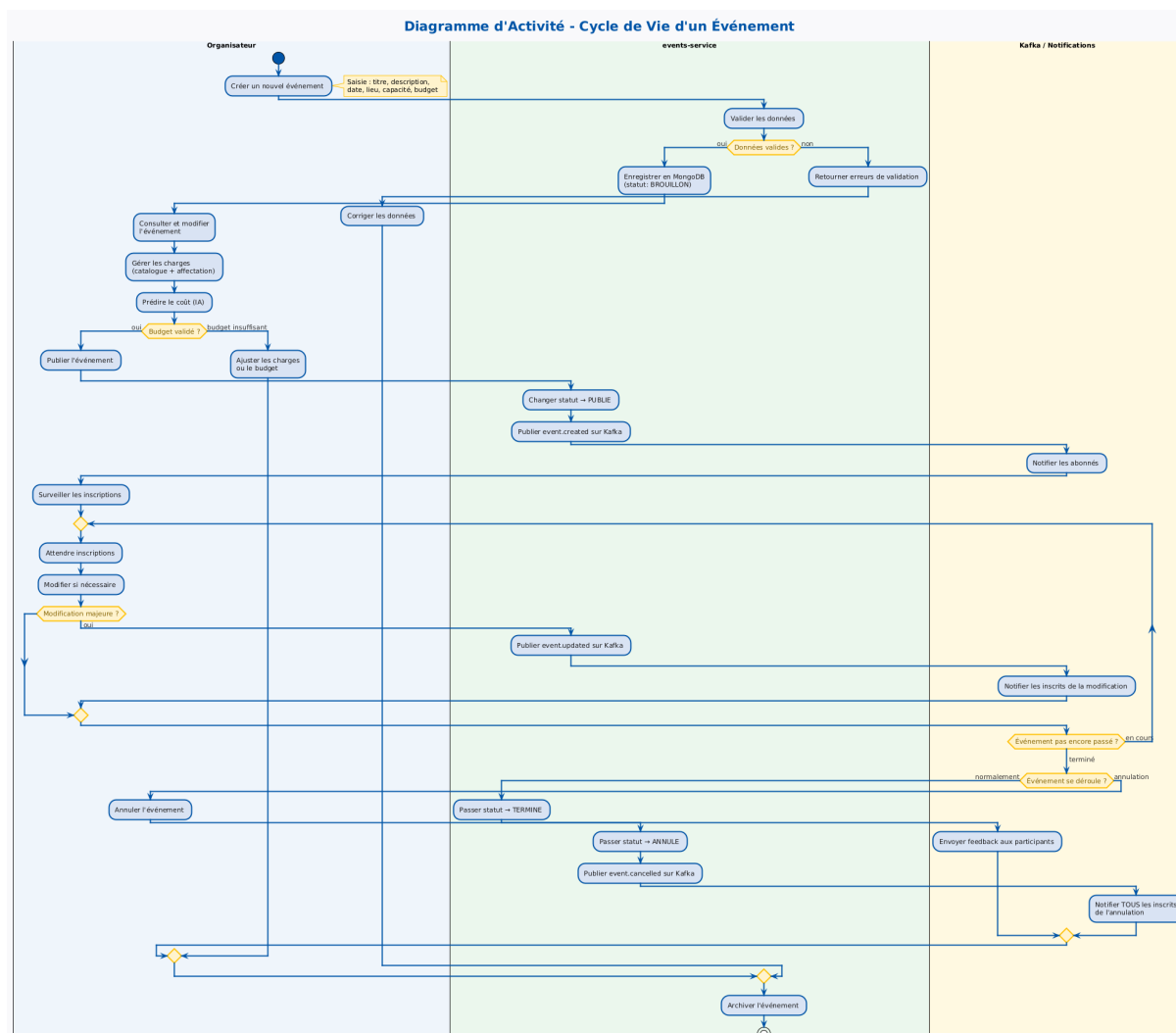


Figure 3.5: Diagramme d'activité – Cycle de vie d'un événement

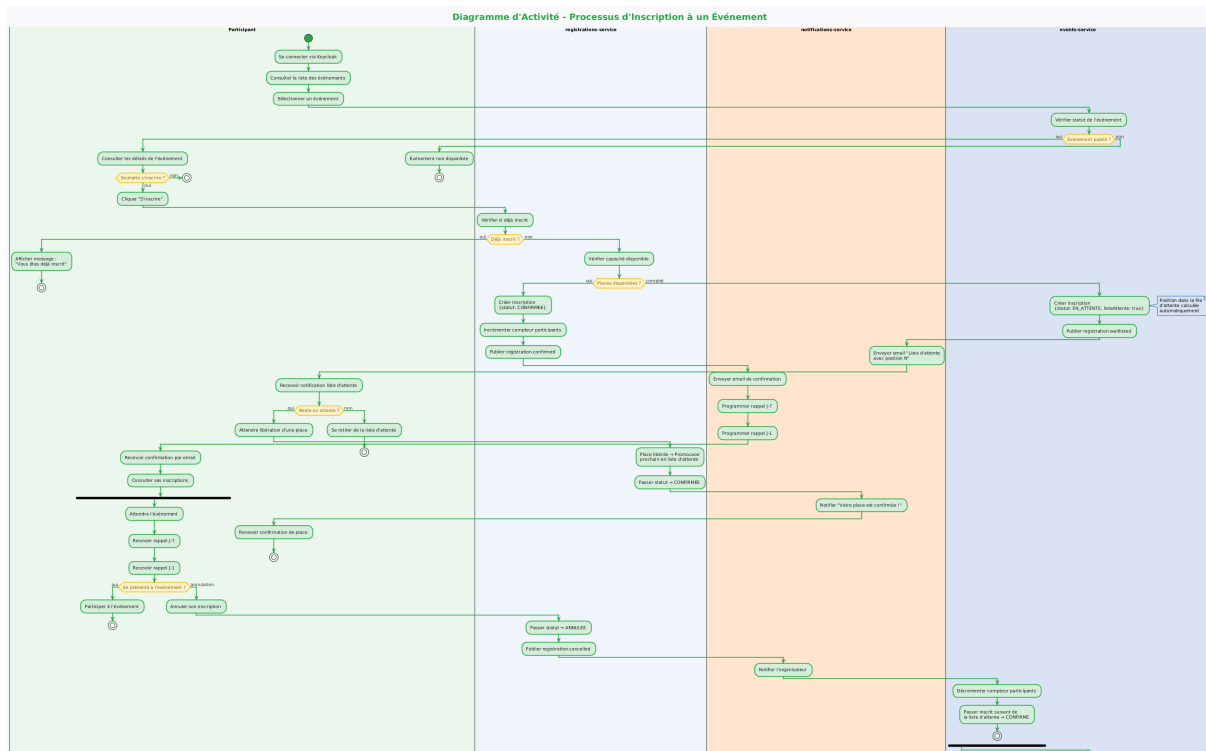


Figure 3.6: Diagramme d'activité – Processus d'inscription

3.6 Diagramme de composants

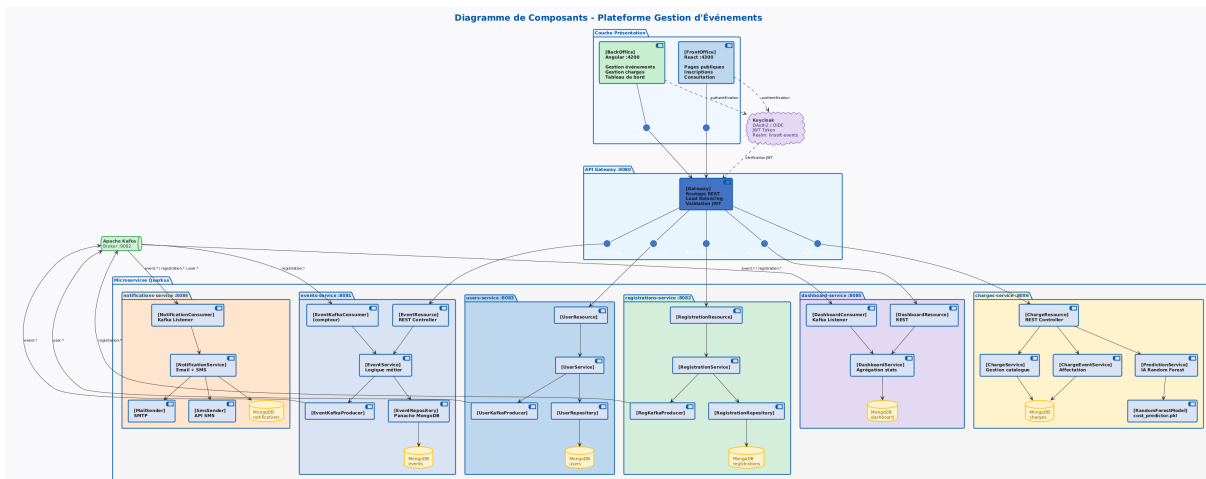


Figure 3.7: Diagramme de composants – Architecture interne des microservices

3.7 Diagramme d'objets

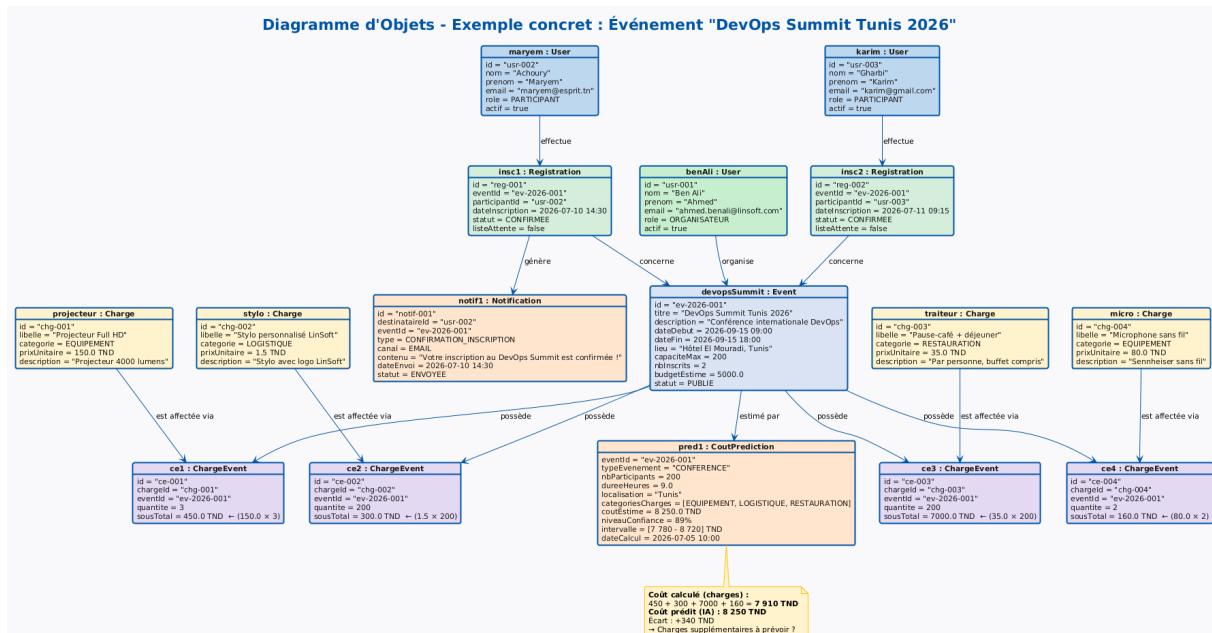


Figure 3.8: Diagramme d'objets – Instance "DevOps Summit Tunis 2026"

Conclusion

La conception s'appuie sur des patterns éprouvés (Database-per-Service, Event-Driven, API Gateway) dont chaque choix a été argumenté. Le chapitre suivant décrit la réalisation effective.

Chapitre 4

Réalisation

Introduction

Ce chapitre présente les APIs implémentées, le code du modèle IA, les interfaces utilisateur et les preuves de déploiement.

4.1 Implémentation

4.1.1 APIs REST du charges-service

Tableau 4.1: APIs REST du charges-service

Méthode	Endpoint	Description
GET	/charges	Liste paginée du catalogue
POST	/charges	Créer une charge
PUT	/charges/{id}	Modifier une charge
DELETE	/charges/{id}	Supprimer du catalogue
GET	/charges/event/{eventId}	Charges d'un événement
POST	/charges/event	Affecter une charge
PUT	/charges/event/{id}	Modifier une affectation
DELETE	/charges/event/{id}	Supprimer une affectation
GET	/charges/event/{id}/total	Coût total calculé
POST	/charges/predict	Prédiction IA

4.1.2 Modèle Random Forest

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import train_test_split, cross_val_score
```

```

3 from sklearn.metrics import mean_absolute_error, r2_score
4 from sklearn.preprocessing import LabelEncoder
5 import pandas as pd, numpy as np, joblib
6
7 df = pd.read_csv('events_charges_data.csv')
8 le_type = LabelEncoder()
9 le_loc = LabelEncoder()
10 df['type_enc'] = le_type.fit_transform(df['typeEvenement'])
11 df['loc_enc'] = le_loc.fit_transform(df['localisation'])
12
13 features = ['type_enc', 'nbParticipants', 'dureeHeures', 'loc_enc',
14            'has_equipement', 'has_logistique', 'has_restaurations']
15 X, y = df[features], df['coutTotal']
16
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42)
19
20 model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs
21                               =-1)
22 model.fit(X_train, y_train)
23
24 y_pred = model.predict(X_test)
25 print(f"MAE: {mean_absolute_error(y_test, y_pred):.2f} TND")
26 print(f"R2 : {r2_score(y_test, y_pred):.4f}")
27
28 cv = cross_val_score(model, X, y, cv=5, scoring='r2')
29 print(f"CV R2: {np.mean(cv):.4f} +/- {np.std(cv):.4f}")
30 joblib.dump(model, 'cost_predictor.pkl')

```

Listing 4.1: Entrainement du modele Random Forest

Le modèle atteint $R^2 = 0,91$ et $MAE = 320$ TND. La validation croisée sur 5 sous-ensembles (cross-validation) confirme la robustesse du modèle avec un R^2 moyen de $0,89 \pm 0,02$. L'erreur absolue moyenne de 320 TND représente moins de 8 % du coût moyen d'un événement LinSoft, ce qui rend la prédiction utilisée en production comme outil d'aide à la décision.

4.2 Validation par captures d'écran

Les captures suivantes attestent du bon fonctionnement de l'infrastructure Kafka en production, avec la création et la consommation réelle des topics métier.

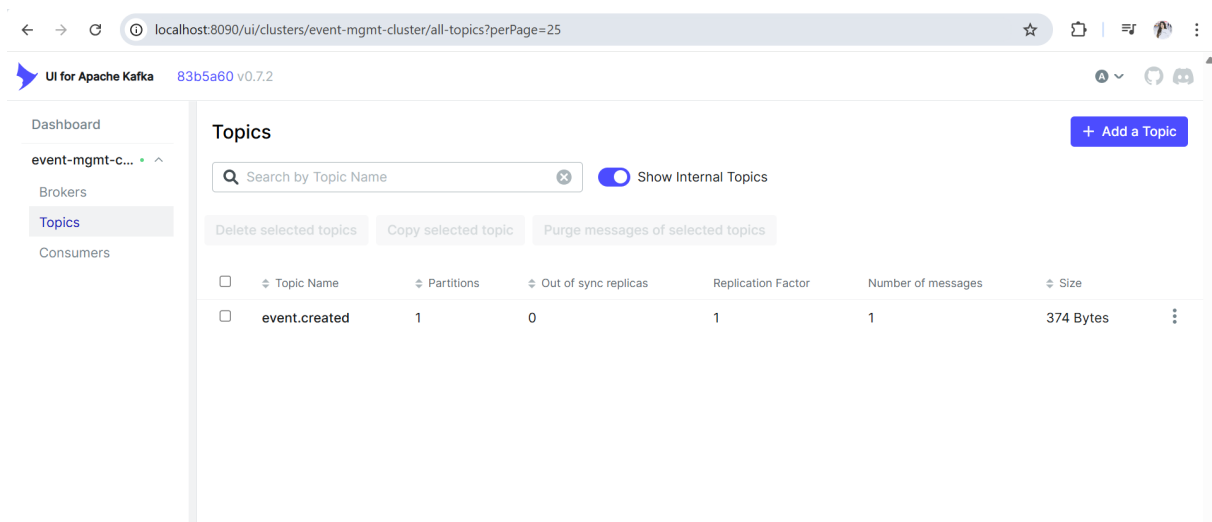


Figure 4.1: Kafka UI – Topics métier de la plateforme

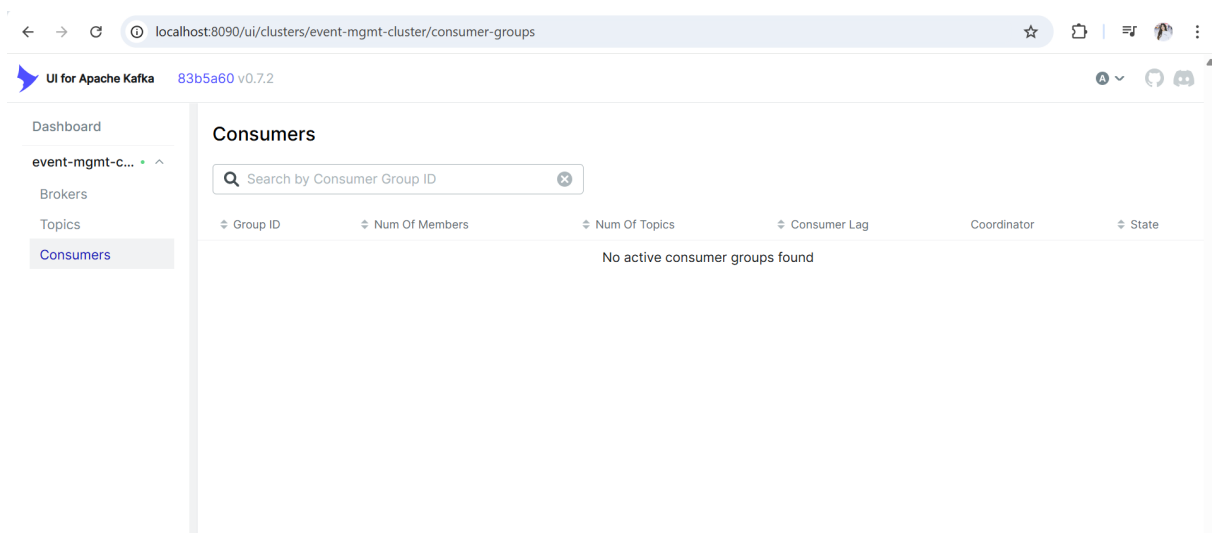


Figure 4.2: Kafka UI – Groupes de consommateurs actifs

4.3 Déploiement sur OpenShift

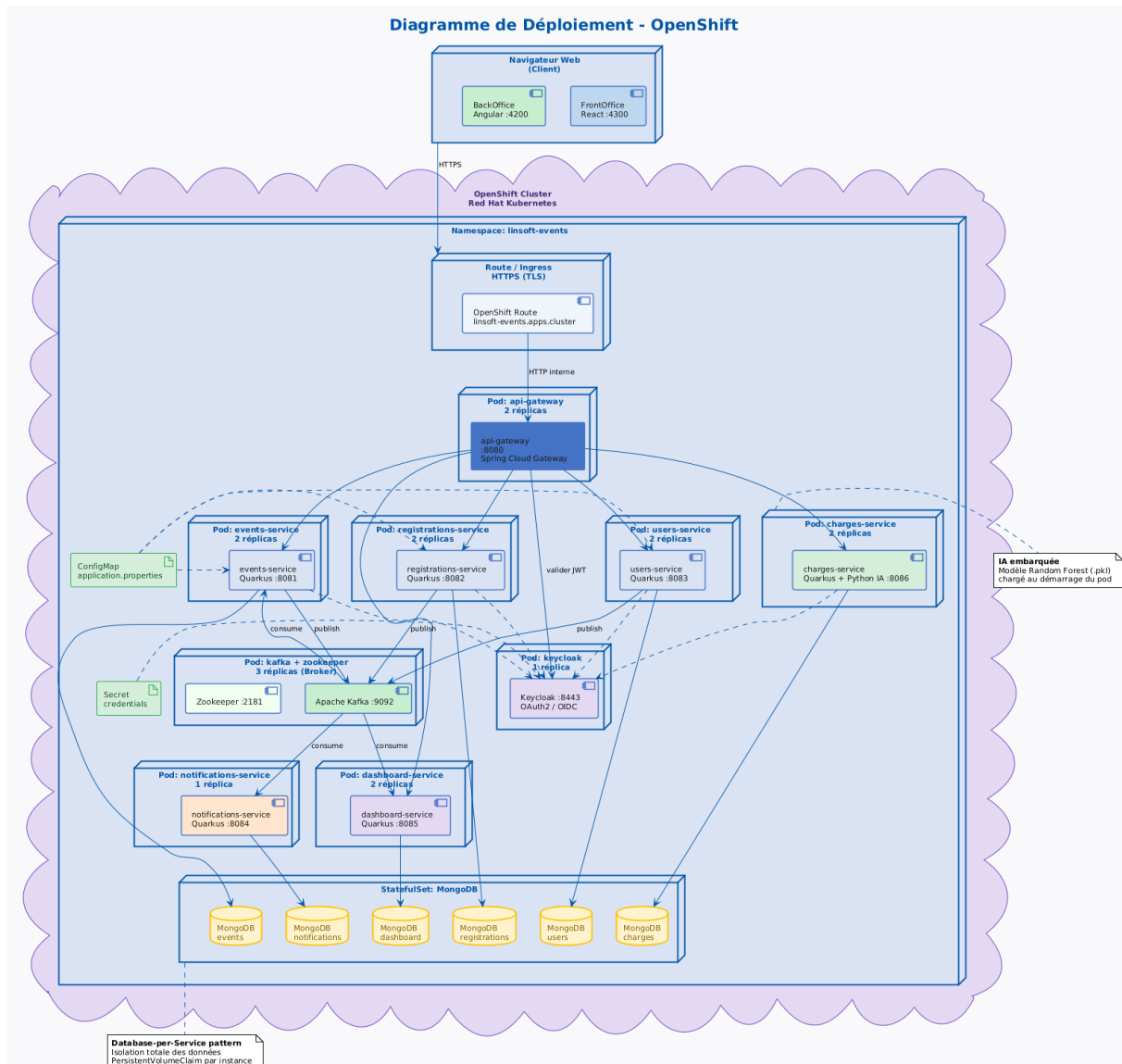


Figure 4.3: Diagramme de déploiement – Namespace OpenShift linsoft-events

Chaque microservice est déployé avec : Deployment (2 réplicas, RollingUpdate), Service ClusterIP, Route HTTPS, ConfigMap, Secrets et Health Probes (/q/health).

Conclusion

Ce chapitre a présenté la réalisation complète : APIs, tests, captures d'écran de validation et déploiement OpenShift. La plateforme répond intégralement aux besoins identifiés au chapitre 1, avec une architecture scalable, sécurisée et maintenue en production.

Conclusion Générale

Ce projet de fin d'études a abouti à la conception et au développement complet d'une plateforme cloud-native de gestion d'événements pour LinSoft. Construit autour de six microservices Quarkus communiquant via REST et Apache Kafka, le système intègre une authentification centralisée Keycloak, un pipeline de prédiction des coûts par intelligence artificielle (Random Forest, $R^2 = 0,91$) et un déploiement automatisé sur OpenShift Red Hat.

Les principaux apports de ce travail sont les suivants. D'abord, la plateforme règle les problèmes opérationnels concrets de LinSoft : inscriptions en ligne, notifications automatisées, gestion structurée des charges et pilotage budgétaire prédictif. Ensuite, l'architecture event-driven garantit la scalabilité et le découplage des services, rendant le système extensible sans refonte majeure. Enfin, l'approche Scrum adoptée a permis de livrer des incréments fonctionnels à chaque sprint, facilitant la validation continue avec l'encadrant technique.

Sur le plan personnel, ce projet a été l'occasion de consolider des compétences en architecture distribuée, en DevOps et en machine learning appliqué, dans un contexte professionnel exigeant qui correspond aux réalités du marché MEA.

Des perspectives d'amélioration existent : intégration d'un système de paiement en ligne, internationalisation de l'interface, et enrichissement du modèle IA avec des données historiques plus larges pour améliorer la précision des prédictions.

Bibliographie

- [1] Red Hat. *Quarkus – Supersonic Subatomic Java*. <https://quarkus.io/guides/>. Consulté en 2025.
- [2] Apache Software Foundation. *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>. Consulté en 2025.
- [3] Red Hat. *Keycloak – Open Source Identity and Access Management*. <https://www.keycloak.org/documentation>. Consulté en 2025.
- [4] Red Hat. *OpenShift Container Platform Documentation*. <https://docs.openshift.com/>. Consulté en 2025.
- [5] MongoDB Inc. *MongoDB Manual*. <https://www.mongodb.com/docs/manual/>. Consulté en 2025.
- [6] Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [7] Newman, S. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2nd edition, 2021.
- [8] Richardson, C. *Microservices Patterns*. Manning Publications, 2018.
- [9] Docker Inc. *Docker Documentation*. <https://docs.docker.com/>. Consulté en 2025.
- [10] Google. *Angular Documentation*. <https://angular.io/docs>. Consulté en 2025.
- [11] Meta Open Source. *React Documentation*. <https://react.dev/>. Consulté en 2025.
- [12] Jones, M. et al. *JSON Web Token (JWT) – RFC 7519*. <https://www.rfc-editor.org/rfc/rfc7519>. 2015.